# SortBot

Authors: Teddy Lin, Ethan Lu, and Alejandro Miranda
Affiliation: Electrical and Computer Engineering, Carnegie Mellon University

***Abstract*—A system capable of sorting recyclables from trash within waaste management facilities. Our system is a robotic gantry that moves along the x, y, and z-axis to pick up trash, using a vacuum-based end effector to pick up the trash, identify it using computer vision, and place it into its corresponding bin. Additionally, our system provides a sorting accuracy of 90% via autonomous means.**

***Index Terms*—3-Axis Gantry, Computer Vision, Machine Learning, Oriented Bounding Boxes, Recycling, Trash, Vacuum, Web Application, YOLO**

## 1 INTRODUCTION

Many current trash sorting solutions remain largely ineffective in sorting all of the different types of trash without the involvement of manual human sorting. In providing a more cost-effective and efficient solution, we have designed SortBot, a trash sorting robot that aims to eliminate the need for manual human sorting.

Our project aims to aid in the processing of the large amount of waste that gets passed through waste sorting facilities every day. The need for properly sorting recyclables out of general trash is essential to address the near 2.12 billion tons of waste produced each year across the world [6]. To add to the severity of this staggering statistic, the USA currently recycles only 32.1% of its trash, which means that 69.9% of its trash is incinerated or lost somewhere in the environment [1].

Our product is meant to be used by waste management facilities, as many currently employ human workers who have to stand by the conveyor belts and manually sort the trash using their hands. It has been shown the exposure to waste products in this line of work can cause health issues [15]. Our robot operates above the conveyor belt and autonomously does the same task as the human workers.

One competing technology is that of ZenRobotics, a company that employs the use of a gripper type end effector on a gantry type system. The gripper is limited in the types of objects it can manipulate, as the objects must be of a certain size [16]. Our suction-based end effector, however, is not impacted by the size of the object, as long as it is under a certain weight. Another competing technology is that from Recycleye, which uses a robotic arm with a suction-based end effector [10]. Despite the dexterity of a robotic arm with a gripper-type end effector, our solution provides users with a linear gantry which is able to avoid the movement limits associated with the inverse kinematics of a robotic arm. This is not to say that our system is better overall, it is just an exploration into a different niche of automated trash sorting. Both of these companies provide robots that operate faster than our solution, with higher sorting rates.

Our goals in implementing this system are to provide an autonomous and seamless trash sorting experience over conveyor belts for waste management facilities. We also intend on achieving a sorting accuracy of 90%, making our project comparable to that of the human manual laborers and potentially superior to them. With SortBot, we aim to create a cleaner and more eco friendly society.

## 2 USE-CASE REQUIREMENTS

The use-case requirements for our project can be divided into three major components, each crucial to our project's overall success.

### 2.1 Reliable Sorting

The end effector should be able to both pick up and release objects at the correct times with 95% accuracy. Our system uses a suction cup to pick up items that will be lowered and raised so that objects that are adjacent to each other (but not overlapping) can be sorted without issue. The end-effector should be able to deal with common recyclable items. This means it should exert a force strong enough to lift objects weighing up to 1 lb, and should be able to handle objects that range from 1 in in their shortest dimension to objects that are 8 in in their longest dimension. Additionally, the end-effector must be able to deal with objects that have a wide variety of surfaces, such as smooth, porous, or rough surfaces.

### 2.2 Real-time Monitoring

As for the monitoring aspect of the system, we want to provide a live stream through a web application. The purpose of this is the ensure the safety of our robot and those around it. In the case that hazardous materials such as batteries, sharp objects, or other combustible chemicals a human supervisor can stop the robot and the conveyor belt with the press of a button. This can prevent the hazardous material from being processed and potentially damaging the robot or causing a fire in the facility. Additionally, the monitoring should help workers confirm that the robot is properly sorting recyclables, as a drop in performance has huge impacts on those who use the recycled materials [13]. Thus, the web application should provide real-time monitoring with annotations through bounding box drawings on detected objects. The bounding box should appear in four

different colors depend on their trash type: red for plastic, green for paper, blue for metal, and purple for glass. These visual updates should happen within 50 ms of the bounding box metadata. Moreover, this live stream should have a resolution of 1280x720 pixels and a frame rate of 10 fps to ensure a decent enough viewing experience on most monitors.

### 2.3 Real-time Object Detection

The system is designed to provide continuous, real-time object detection for objects moving slowly down a conveyor belt. Since waste management facilities are chaotic, our system must support multi-object detection with up to 10 items per image. In addition, the system must be able to detect these objects within 150 ms of their appearance in the camera feed, so that the system can determine the object's location regardless of how fast the conveyor belt is moving. The detection will ensure that the corresponding bounding box for each object successful lands on the object with a tight fit.

## 3 ARCHITECTURE AND/OR PRINCIPLE OF OPERATION

As you can see in Figure 1, we use the eYs3D stereo camera to capture RGB images of the objects. The RGB image is sent to a bounding box classification model to identify the objects and provide their location. This information, along with the heights of the objects, is sent serially as XYZ coordinates to an Arduino Mega. This Arduino Mega then controls a robotic gantry to move over to the objects, pick them up, and place them into the correct bins. The end-effector of this gantry uses a suction-cup-shaped end effector that pulls a vacuum to pick up objects via a vacuum pump. The video, as well as the objects and their bounding boxes, is sent to a web app for users to monitor the robot's operation.

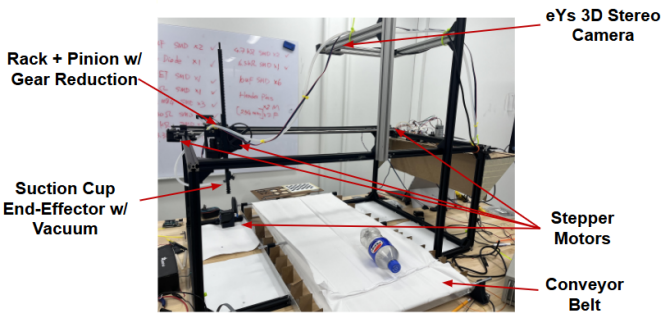


Figure 1: Overall Gantry Diagram

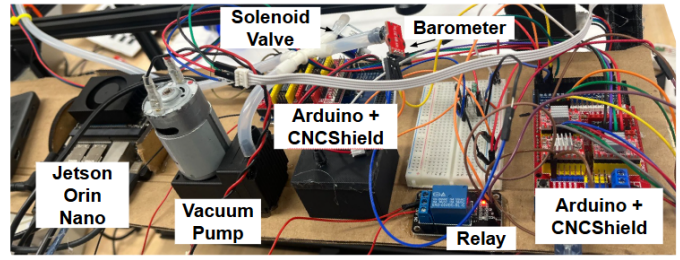


Figure 2: Picture of SortBot's Hardware

## 4 DESIGN REQUIREMENTS

Each of our three primary use-case requirements can be translated into multiple design requirements to support them. In this section, we will break them down by use-case requirement.

### 4.1 Reliable Sorting

For our requirement, the system must be able to pick up and move the objects to the correct location reliably and within an acceptable time frame. After the model has identified the centroid, the time that the gantry takes to sort an item should be no more than 8 seconds. Ideally, the gantry should take 2 seconds maximum to move to the centroid, 4 seconds to pick it up, and 2 seconds to move the object above the correct bin and drop it. Additionally, in order to ensure that the robot can move to the correct location to pick up the object, the gantry should be able to move in 0.2 mm increments on the x-y axes and in 1 mm increments in the z axis. In order to manipulate items as small as 1in, the diameter of the suction cup should be no larger than 1in. Additionally, the vacuum pump should be able to hold a vacuum of at least 1 lb and evacuate air quickly enough in order to lift objects that have a porous or rough surface.

### 4.2 Real-time Monitoring

For our requirement of having real-time monitoring, we are able to derive some more design requirements. In order to achieve a 50 ms second latency, each component: network transmission, and browser rendering must add up to less than 50 ms. The network transmission should take no more than 10 ms, and finally the browser rendering should take no more than 15 ms. Together this all adds up to 45 ms. When testing the network speed on the CMU-SECURE network, we achieved a 83.2 Mbps upload speed, with a raw data rate of:

$$720 \times 720 \times 3 \text{ bytes} \times 30 \text{ fps} = 46.66 \text{ MB/s}$$

This leaves us below the 83.2 Mbps available upload bandwidth: the bandwidth utilization is $46.66/83.2 = 56.08\%$. This was the initial thought process for our original implementation of data streaming. We have now changed it so

that the video format is of 1280x720p for a more standard viewing experience. Additionally, decided to drop the fps to 15 fps to give the system slightly more overhead to accommodate the standard resolution. So this is math for the new upload speed:

$$1280 \times 720 \times 3 \text{ bytes} \times 15 \text{ fps} = 41.47 \text{ MB/s}$$

This now leaves us further below the 83.2 Mbps available upload bandwidth: the bandwidth utilization is 41.47/83.2 = 49.84%.

## 4.3 Real-time Object Detection

For our requirement of having real-time object detection, we are able to derive some more design requirements. We want the system to consistently identify objects of the same type in the same way, to minimize false positives, thereby reducing waste contamination. We are requiring a precision of 0.70 for this. Moreover, the identified bounding boxes must line up properly with their ground truths, in order for the gantry to pick the object. A mIoU of 0.95 would allow the gantry to be within ± 5 pixels of the true bounding box centroid.

# 5 DESIGN TRADE STUDIES

## 5.1 Mechanical Movement

In order to physically manipulate the objects in 3D space, there needs to be a mechanical system that can move the objects from the conveyor belt to the bins.

### 5.1.1 Articulated Robotic Arm

Initially, we considered using an articulated robotic arm to pick and place objects. These arms are often used in production lines as they are able to move with 6+ DoF, meaning that they can also control the orientation of their end-effector as well as its position. However, these systems are often costly, since they require motors and parts that are able to withstand high torques and jerks. Additionally, these arms are restricted by their joint limits, and require the use of inverse kinematics in order to move from one place to another. This means that moving between certain configurations will require additional time as it moves to more suitable configurations, or else the movement becomes impossible.

### 5.1.2 End-of-Conveyor Ramp

One potential way of sorting the objects into the correct bins is to put a moving ramp that is able to adjust itself, such that it guides objects that fall onto it into the correct bin. This implementation is able to deal with heavy objects, since it doesn't need to lift anything off the conveyor belt, and instead uses the force of gravity to move objects into their respective bins. However, one downside to this implementation is that it doesn't work with objects that are next to each other, as they will fall onto the ramp at the same time, so the ramp is forced to sort two potentially different types of trash into the same bin.

### 5.1.3 3-Axis Gantry

A robotic gantry (otherwise known as a Cartesian Coordinate or Linear robot) which controls its motion through linear motion in each respective axis. These types of robots are used in a wide variety of applications, such as pick-and-place, welding, plotting, and 3D printing. They are highly precise, and since the movement in each direction is linear, they are able to move to any location that falls within the extremes of each linear motion (which ends up being a rectangular prism). This type of motion is also much simpler, and does not require the use of inverse kinematics. Although these robots only have 3 DoF (x,y,z), it is enough for our application, since the robot only has to move the end-effector down on top of the objects without any rotation.

## 5.2 End-Effector

To pick up the objects reliably, the system will need some way to grab on to and release the trash so it can be moved from the belt to the bins.

### 5.2.1 Claw Gripper

One common end-effector design is a claw, where two finger-like appendages are closed around an object to grasp it. This design can do well with heavy objects, as it has the ability to dig into objects as it clamps down, making it hard for them to slip out. However, it has difficulty picking up objects that are larger than its grip, and in particular struggles with wide, flat objects like cardboard and paper.

### 5.2.2 Suction Gripper

Another end-effector design involves a suction cup with an additional vacuum pulled through its center, which uses negative pressure to keep objects adhered to it. While this design struggles a bit with heavy objects, as well as ones with porous textures, it excels in picking up a wide variety of object shapes and sizes, including wide, flat ones like cardboard and paper.

## 5.3 Web Application Architecture

In order to stream video with sufficient frames per second, delay, and resolution, the web application must provide a software architecture framework suitable for video streaming.

#### 5.3.1 Django

Django is a Python web application framework. Initially, we considered using the Django web framework for creating the web app and streaming the video. Despite it having various tools useful for account management purposes, such as in the case of a social media application, there is a simpler and better alternative for our design requirements: Node.js [8].

#### 5.3.2 Flask

Flask is a lightweight web application framework written in Python. Its main benefits are the fact that it's lightweight, which makes it perfect for devices with limited resources. Additionally, Flask was designed to be simple and easy to use. Despite these features Flask provides, Node.js remains the optimal choice due to it's "its excellent performance due to its non-blocking I/O model and event-driven architecture" [3].

#### 5.3.3 Node.js

Node.js is a JavaScript web app framework. This backend framework is favored by technology companies due to its ability to handle data streaming efficiently [2]. This is partly due to the fact that Node.js handles data "processes data in chunks using streams rather than buffering the entire data in memory, resulting in faster processing and reduced memory consumption," which leads to an overall better experience with real-time data like video streaming [17]. Despite lacking the simpler experience Django provides for the integration of a database and the native Python language, or the overall simpler design experience of Flask, Node.js serves us slightly better in achieving our design requirements.

### 5.4 Deep Learning Model Architecture

Object detection models are categorized into two types: one-stage and two-stage, based on their approach to the detection pipeline. One-stage models treat object detection as a single unified task, meaning that they predict bounding boxes and class probabilities as a single unified task. On the other hand, two-stage models first generate a set of region proposals during the first stage, and then during the second stage, the model refines these proposals by classifying the objects and fine-tuning the predicted bounding boxes. While two-stage models tend to be more precise due to their more thorough approach, they are generally much slower and require significantly more computational resources than their counterparts. One-stage models fit our use-case much more, as we are willing to sacrifice a bit of precision for a faster inference.

**Single Shot Multibox Detector** The Single Shot Multibox Detector (SSD) is an approach proposed by Wei et al. [5] in 2016 during his time as a Ph.D. student at the University of North Carolina at Chapel Hill. SSD discretizes the output space of bounding boxes into a set of default bounding boxes over different aspect ratios and scales per feature map location. At prediction time, the network generates scores for the presence of each object category in each default box and produces adjustments to the box to better match the object shape. The refinement process is contingent upon the predefined default bounding boxes at each feature map location. While this approach may be effective for many scenarios, it can become a problem when objects appear in unusual sizes, orientations, and locations. Since the refinement process uses a naive approach of using predicted offsets in xy position and width and height values to adjust the default bounding boxes, it is possible that SSD can still fail to properly detect objects in the previously mentioned scenarios. Without guarantees on object size, orientation, and location, SSD is not the right approach for this problem.

**EfficientDet** The EfficientDet architecture is a family of object detectors proposed by Tan et al. [12] in 2019 during his time on the Google Deep Brain team. As opposed to previous state of the art approaches, EfficientDet introduces a new type of feature extractor, the a weighted bi-directional feature pyramid network, for fast multiscale feature fusion, and a compound scaling method that uniformly scales the resolution, depth, and width for all backbone, feature networks, and box/class prediction networks all at the same time. While EfficientDet takes a more thorough approach to bounding box localization by using repeated top-down and bottom-up feature fusion, this level of refinement is excessive for our purposes. Since we are allowing a margin of error of $\pm 5$ pixels for computed bounding box centroids, dedicating computational resources for finer localization would introduce unnecessary overhead, and would ultimately degrade real-time performance. The additional computational resources needed to take full advantage of the EfficientDet architecture are not justified in our use case, where it is acceptable to sacrifice some precision in bounding box localization if it can lead to significant improvements to real-time performance.

**You Only Look Once** The You Only Look Once (YOLO) architecture is a family of object detectors proposed by Redmon et al. [11] in 2016 during his time as a Ph.D. student at the University of Washington. As opposed to previous state of the art approaches, YOLO frames object detection as a regression problem to spatially separate bounding boxes and associated class probabilities. As opposed to other state of the art approaches, YOLO uses a single network to predict bounding boxes and class probabilities. Since the whole detection pipeline fits inside of a single network, YOLO has been optimized end-to-end directly for detection performance. YOLO's appeal comes from its comparable performance to other state of the art approaches despite using a fewer number of parameters. The following Performance Comparison Table 1 between

the YOLO architecture and the EfficientDet architecture clearly illustrates this fact. A YOLOv8n model with 3.2M parameters has a greater mAPVAL on Microsoft COCO [4], a common object detection benchmark, and faster inference speed on a T4 TensorRT10 (which we can extrapolate to our GPU-based hardware) than its 3.9M parameter EfficientDet counterpart, EfficientDet-d0. This trend continues for YOLO sizes and their EfficientDet counterparts. The strong real-time performance without loss of precision is what makes YOLO very attractive to us.

#### 5.4.1 Classification Head

Object detection is a task in deep computer vision that involves identifying the location and the class of objects in an image or video stream. Objects that are found are given a bounding box, typically represented with an xy coordinate that represents the center of the detected object and a width and height value that is used to indicate the size of the object. These values are inferred from the last layer of object detection models, which is referred to as the classification head. The effectiveness of an object detection model heavily depends on its classification head. In order to select a classification head that we can trust to meet our performance requirements, we will need to inspect how each type of bounding box will be visualized on static images.

#### 5.4.2 Axis-Aligned Bounding Box Prediction

Object detection models trained for axis-aligned bounding box prediction infer a detected object's location using the *(x, y, w, h)* format where $x$ and $y$ are the predicted coordinates for the center of the bounding box and w and h are predicted width and height respectively. As illustrated in Fig.3, we can see that axis-aligned bounding boxes are not very space-efficient, resulting in a less accurate representation of an object's true shape and location.

#### 5.4.3 Oriented Bounding Box Prediction

Object detection models trained for oriented bounding box (OBB) prediction infer a detected object's location using the *(x, y, w, h, θ)* format where *x, y, w, and h* represent the same values as above, and $\theta$ is the angle of rotation. As illustrated in Fig.3, we can see that oriented bounding boxes provide a much tighter fit. Moreover, we can see that the calculated bounding box centroid lies closer to the true centroid than the axis-aligned bounding box equivalent.

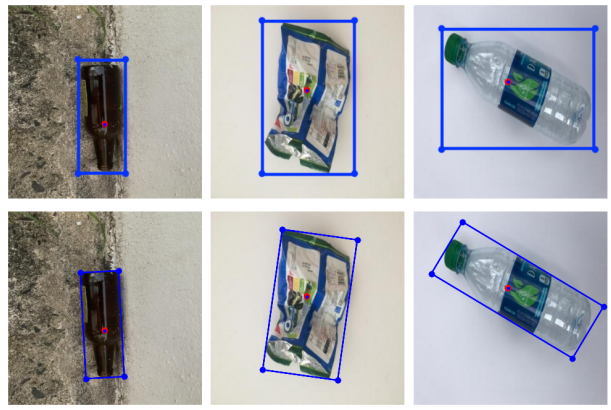

Figure 3: Visual Comparison of Axis-Aligned and Oriented Bounding Boxes on Validation Dataset Images

#### 5.4.4 Device for Central Computing

Being able to achieve on-device deep learning is a key component in being able to achieve our design requirements as local compute not only minimizes the number of external data transfers, which will reduce overall latency, but can also provide the necessary control to perform further on-device optimizations.This requirement dictated our decision to centralize the deep learning model on a dedicated device, which is responsible for both running the model and managing data communication between the web application and the two Arduino Megas.

#### 5.4.5 Nvidia Jetson Nano

The Jetson Nano is a mid-range mobile graphics chip that was released by Nvidia in 2019. Built on the 20 nm process and based on the TM660M-A2 variant of the GM20B graphics processor, the Jetson Nano offers strong computational power relative to its cost of $99. The Jetson Nano offers an AI performance of 472 GFLOPS and a GPU and CPU max frequency of 921 MHz and 1.42 MHz, respectively. These specifications will not be sufficient to run modern deep computer vision models, which typically require a GPU frequency of 1 GHz.

#### 5.4.6 Nvidia TX2 Jetson

The Jetson TX2 is one of the direct successors to the Jetson Nano. Jetson TX2 can deliver up to 2.5x the performance of the Jetson Nano, offering the AI performance 1.33 TFLOPS and a GPU and CPU max frequency of 1.3 GHz and 2.2 GH,z respectively. Despite having attractive properties in terms of raw throughput, the Jetson TX2 suffers when it comes to its memory constraints. The Jetson TX2 only has 8GB of LPDDR4 RAM, which is unideal in our use case. Large models and high-resolution inputs will create memory bottlenecks which will lead to slower performance than advertised.

| Model | $mAP^{val}$ | CPU ONNX (ms) | T4 TensorRT10 (ms) | Parameters (M) | FLOPS (B) |
|---|---|---|---|---|---|
| EfficientDet-d0 | 34.6 | 10.2 | 3.92 | 3.9 | 2.54 |
| EfficientDet-d1 | 40.5 | 13.5 | 7.31 | 6.6 | 6.1 |
| EfficientDet-d2 | 43.0 | 17.7 | 10.92 | 8.1 | 11.0 |
| EfficientDet-d3 | 47.5 | 28.0 | 19.59 | 12.0 | 24.9 |
| EfficientDet-d4 | 49.7 | 42.8 | 33.55 | 20.7 | 55.2 |
| EfficientDet-d5 | 51.5 | 72.5 | 67.86 | 33.7 | 130.0 |
| EfficientDet-d6 | 52.6 | 92.8 | 89.29 | 51.9 | 226.0 |
| EfficientDet-d7 | 53.7 | 122.0 | 128.07 | 51.9 | 325.0 |
| YOLOv8n | 37.3 | 80.4 | 1.47 | 3.2 | 8.7 |
| YOLOv8s | 44.9 | 128.4 | 2.66 | 11.2 | 28.6 |
| YOLOv8m | 50.2 | 234.7 | 5.86 | 25.9 | 78.9 |
| YOLOv8l | 52.9 | 375.2 | 9.06 | 43.7 | 165.2 |
| YOLOv8x | 53.9 | 479.1 | 14.37 | 68.2 | 257.8 |

Table 1: Comparative analysis of YOLOv8 and EfficientDet [14] performance using 640x640 input images. These results were used to guide exceptions for our 416x416 input images.

#### 5.4.7 Nvidia Orin Jetson Nano

The Orin Jetson Nano is one of the more modern graphics chips that has come out from Nvidia. Released in 2023, the Orin Jetson Nano offers a promising 67 TOPS and a GPU and CPU max frequency of 1.2 MHz and 1.7 GHz. Unlike its older counterparts the Orin Jetson Nano uses 8 GB 128-bits LPDDR5. Compared to the LPDDR4, LPDDR5 has a high data transfer rate, boosting a maximum bandwidth of 6400 Mbps, which is nearly 1.5 times faster than the LPDDR4's 4266 Mbps. These specifications, combined with the fast data transfer rates, make the Orin Jetson Nano very attractive for our use case.

# 6 SYSTEM IMPLEMENTATION

## 6.1 3-Axis Gantry

### 6.1.1 XY movement

For the xy movement of the gantry (parallel to the plane of the surface of the conveyor belt), we based our design on the 4xiDraw, a robot made to plot SVG images onto physical paper. This design involves two perpendicular pairs of rods joined by a sliding clamp that slides across the rods in both directions to allow for movement. The movement is driven by a singular belt, powered by two stepper motors (one for each direction). In order to compensate for the stresses caused by leverage when expanding the dimensions of the 4xiDraw design, we built a supporting frame that sat over the gantry to support the end-effector's weight.

### 6.1.2 Z movement

The z-axis movement (perpendicular to the conveyor belt's surface) is controlled by another stepper motor which drives a rack-and-pinion gear system. In order to design the system such that it can support weights up to 1 lb, we added a worm drive with additional gear reductions to raise up the torque, though it came with some reduction in speed.

### 6.1.3 End Effector

The end-effector, which allows the robot to hold objects while they are being moved, is a small suction cup attached to a 12 V vacuum pump via a silicone tube. As the suction cup is be pressed onto the surface of a trash, and the vacuum pump pulls a vacuum inside the cup to maintain the grip. The pressure is then monitored by a barometer to determine when the suction cup has latched onto the object, as the pressure drops when a tight seal is made with the vacuum. A solenoid valve is also attached to the vacuum tube in order to ensure a This end-effector is mounted onto the end of the rack gear, so that it can be lowered and raised.

### 6.1.4 Arduino Control

The stepper motor vacuum pump for the gantry and conveyor belt is controlled by an Arduino Mega 2560. The Arduino is connected to a stepper motor controller board (CNCShield) to control the stepper motors. Since the linear motion is controlled by belts, we can translate translational movement into the number of steps that the motors have to turn:

$$\text{num steps} = \frac{\text{linear distance}}{\pi * \text{pitch diameter}} * (\frac{\text{steps}}{\text{revolution}})$$

As soon as the model detects that a recyclable object is detected and is within the gantry's range, the conveyor belt is stopped. When the gantry has moved to the correct position above the object, the Arduino then sends a signal through a relay in order to activate the 12V vacuum pump, while the end-effector is lowered. As mentioned prior, the Arduino keeps track of the pressure in the vacuum using a barometer to determine when the end-effector has made contact with the object. The Arduino then lifts the end-effector to its original position and moves the gantry over

to the correct bin (which has a predetermined location). In order to ensure that the object is released, there is a

## 6.2 Deep Learning Model

### 6.2.1 Workflow

RGB images captured by the eYs3D stereo camera, which has some geometric distortion. To correct this, we used the camera calibration parameters obtained using the chessboard calibration process outlined in paulmelis/opencv-camera-calibration [9]. The resulting undistorted images are then transmitted to a web application. These raw images are then standardized — using the mean and standard deviation calculated from the training dataset and formula presented Equation 1 — before being resized to 416x416 and then passed to the YOLO-OBB model, which produces the predicted bounding boxes and class labels.

$$X' = \frac{X - \mu}{\sigma} \tag{1}$$

The predicted bounding boxes are then send the web application to be rendered. The centroid of each of the bounding boxes are calculated, and the centroid with the largest y-coordinate (i.e., the one that belongs to the object that is farthest along the conveyor belt) is send to the Arduino Mega that controls the gantry. The coordinates are formatted as "X:{x-coordinate}Y:{y-coordinate}|" with the pipe symbol used as an arbitrary delimiter. See Figure 4 for a visual representation of the workflow.

### 6.2.2 Training

Once the deep learning pipeline has been implemented, we can then transition into training the actual YOLOv8 model. We trained our model using a trash detection dataset from Roboflow [7] after standardizing the background of each image of the dataset to a white color (this decision was made after purchasing the material to make our conveyor belt). We plan to use the initial learning rate of $\eta = 0.001$ to ensure stable training and to avoid overly large updates that may force us out of a global minimum. Moreover, we plan to use the AdamW optimizer and the StepLR learning rate scheduler to help easy our convergence. he combined decoupled weight decay in addition to consistently reducing the learning rate helped the model the better learn the features of each class. We plan to train on Google Colab T4's using the \$300 free credits provided to new users ($3 \times$ \$300 hours = \$900 hours to train). Throughout this project, we trained three differently-sized YOLOv8 models, a YOLOv8-Nano, a YOLOv8-Small, and a YOLOv8-Medium. We found that the YOLOv8-Medium gave the best performance with minimal increase in latency. From here, we were able to perform ablations to determine the best set of hyperparamters to give us the best performance.

# 7 TEST & VALIDATION

## 7.1 Results for Gantry

In order to test the success rate for the entire pickup and drop off, we chose 30 different trash items (8 metal, 7 glass, 8 plastic, 7 paper) and had the gantry do the entire pick-up and drop off sequence for each 3 times, making sure to move them to different locations within the gantry with varying orientations. Out of the 90 total trials, there were 4 failures. One occurred with a plastic bottle which had a label on the side that didn't adhere well enough. The other three occurred with 2 metal and 1 plastic trials, as the objects had a bumpy texture on the side that prevented the suction cup from forming a tight seal. Overall, however, we had a success rate of 95.5% for the entire pick-up and drop off sequence.

In order to ensure that the gantry could lift up objects weighing up to 1lb, we obtained containers from each class and had the gantry lift them, incrementally increased the weight for each trial until pickup failed.

| Object | Class | Max Liftable Weight |
|---|---|---|
| Carboard box | Paper | 0.7 lbs |
| Aluminum Can | Metal | 1 lbs |
| 16.9 oz Soda Bottle | Plastic | 0.9 lbs |
| Glass Soda Bottle | Glass | 1.2 lbs |

Table 2: Results of weight limit tests

Overall, the gantry was able to lift the 1lb glass and metal containers due to their smooth, stiff surfaces, while it had some trouble with paper as the porous surface reduced the pulling force of the vacuum.

To test the speed of the gantry, we ran the longest possible pickup+drop off sequence (starting from the bottom-left, picking up a plastic bottle at the upper-right corner, and dropping it in the plastic bin at the bottom-left), and found that it took a total of 17.89 seconds. This was unfortunately over the 8s longest pickup sequence we were hoping for. The lack of speed is most likely due to the reduction in speed in the z-axis needed to get the torque to lift heavier objects, as well as the hardware limitations of pushing the stepper motors above 1000 steps/sec.

In order to make sure we met the movement accuracy requirement, we moved the gantry a predetermined amount of steps in the x, y, and z direction (since a step is the shortest distance we can move) and measured the distance moved. We then divided this by the number of steps to get that the x-y movement has a $\pm 0.5$mm accuracy, and the z has a $\pm 0.2$mm accuracy (due to the gear reductions). This was fairly close to the $\pm 0.2$mm requirement for xy movement and $\pm 1$mm requirement for z, and proved to be reliable enough to pick up objects consistently.

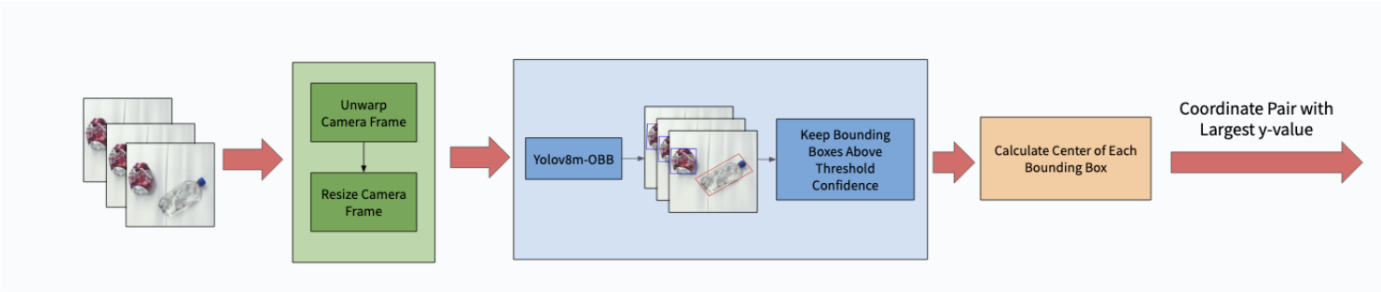


Figure 4: Deep learning pipeline.

## 7.2 Results for Web Application

To test whether the web app was operating under the desired latency we performed various latency tests by recording the round trip time of pings. I recorded timestamps on the Jetson Orin Nano by pinging the web app over the websockets connections and then having the web app send a ping to the Jetson back in order to test the delay in latency time. 30 trials were performed to verify the results, and we achieved an average round-trip time of 50.7ms. Thus, the time for one way communication for the streaming application is 25ms, well below our design requirement.

Additionally, we tested the page load time with another timestamp method, which netted us 190.3 ms over the 30 trials we performed. We have thus reached significantly under the 2-second load time for the web page on the browser we desired to achieve.

Finally, the video resolution was tested by counting the pixels in an image streamed over to the web app. All of these results can be seen listed in Table 3.

| Component | Time |
|---|---|
| Latency | 25.35 ms |
| Page Load Time | 190.3 ms |
| Video Resolution | 1280x720p |

Table 3: Results of Web App Tests

## 7.3 Results for Deep Learning Model

Validation for the performance of the deep learning model was done in two-stages. Stage one used a subset of validation images from the original dataset, while stage two involved a separate set of human-annotated images captured directly on the conveyor belt from the perspective of the camera. The deep learning model achieved classification precisions of 0.72 (using the formula illustrated in Equation 2) on the validation dataset images.

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}} \tag{2}$$

Additionally, it achieved a mIoU of 0.96 on these set of images (using the formula illustrated in Equation 3).

$$mAP = \frac{1}{|classes|} \sum_{c \in classes} \frac{\text{True Positive}_c}{\text{True Positive}_c + \text{False Positive}_c} \tag{3}$$

Stage two of validating the performance of the deep learning model involved creating a small dataset of human-annotated images, specifically captured from the perspective of the mounted camera looking down at the conveyor belt to assess real-world performance. Bounding boxes were manually drawn around the target objects using the Matplotlib Python library (the extract distribution of classes found in this human-annotated dataset is shown in Figure 5). The deep learning model achieved final classification precisions of 0.81 and a mIoU of 0.96.

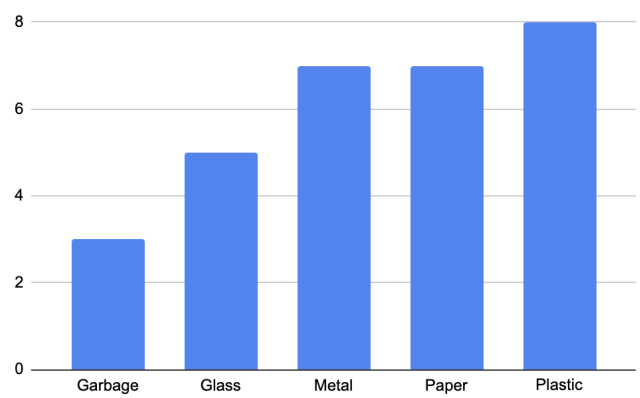


Figure 5: Distribution of classes in the human-annotated dataset of 30 images.

Validation for the inference speed of the deep learning model was done time Python module. The workflow involved compiling the model using just-in-time (JIT) compilation, then running inference 101 times on randomly generated input tensors. The first inference was discarded to account for initialization overhead, and the inference times from the remaining 100 runs were used to compute the average inference speed. The deep learning model achieved an average inference speed of 75 ms.

# 8 PROJECT MANAGEMENT

## 8.1 Schedule

Our schedule is split into four main sections: the xy movement of the gantry, the z-axis movement, the web application, and the deep learning model. Since the design report there have been no major changes to schedule. The full schedule is shown in Figure 7.

## 8.2 Team Member Responsibilities

We have divided team member tasks and responsibilities based on each member's past experience in each field and current interests. Teddy Lin worked mainly on designing the gantry and the custom z-axis end effector that can satisfy the previously mentioned design, as well as the code for calibrating the cameras to gantry positions and the code required for the end-effector's pickup sequence. Ethan Lu worked on implementing a YOLOv8 OBB model and optimizing model performance for image classification accuracy and bounding box mIoU. Alejandro Miranda worked on developing a web stack necessary for real-time live-streaming, conveyor belt speed control, an on/off button, and programming the xy movement of the gantry. The construction of the full system, as well as end-to-end testing and verification, was done together as a team.

## 8.3 Bill of Materials and Budget

The current Bill of Materials for SortBot is shown in Table 4. It includes all currently purchased parts, their quantities, vendors, and final prices.

## 8.4 TechSpark Usage

We used the TechSpark Machine Shop to cut the aluminum extrusion and their tools (e.g. drills) to assembly the gantry system.

## 8.5 Risk Management

### 8.5.1 Suction-Type End-Effector

When we originally designed the gantry, we anticipated that one of the riskier aspects of the design would be the suction cup's ability to adhere to objects of different surfaces. We thus came up with a backup plan to attach a vacuum pump to the cup if it did not perform well enough. We tested the suction cup's performance early on in the semester, and so we were able to determine that the vacuum was needed fairly early on in the project schedule.

### 8.5.2 Depth Estimation

We also acknowledged that our depth estimation for determining how much to lower the end-effector to pick up the object might be a source of risk, but didn't anticipate that it would be as much of a problem. However, later in the semester, we discovered that the API for the camera we were using was not well established, and we had trouble getting it up and working. Anticipating that it would take too long to get it working, we pivoted to one of our fallback plans to use a ToF distance sensor on the end-effector, which could tell the Arduino Mega when the end-effector was close enough to interface with the object. However, this also proved not to be reliable, as we were not able to position the distance sensor directly under the end-effector, and the curved surfaces of some of the objects meant that reading it would get shape when the end-effector had met the object would depend on the object's shape. As a final fix, we changed our approach by incorporating a barometer that would detect drops in pressure to indicate that the end effector had met the object, instead of trying to gauge distance. This approach was successful.

# 9 ETHICAL ISSUES

## 9.1 Public Health

With respect to public health, SortBot, as an autonomous recyclable sorting robot for waste management facilities, can positively impact public health by decreasing the amount of waste contamination, which would in turn lower the amount of environmental pollution than the current solution (using human workers). While SortBot can process waste autonomously, it may still lack the nuanced judgment and decision-making skills of a trained worker. To mitigate this risk, we could add a feature on our web application where the user can change classifications labels. This way, human workers can stay away from breathing in toxic chemicals and still use their keen decision-making skills to decrease waste contamination.

## 9.2 Public Safety

With respect to public safety, SortBot can reduce the risk workers put themselves in when working in a waste management facility. While working in these facilities, workers run the risk of cutting themselves on foreign objects and breathing in toxic chemicals. Meanwhile, using SortBot would reduce these risks significantly. Like all systems, however, SortBot ruins the risk of breaking down in the future, and accidental injuries from this may be a problem. To mitigate this risk, we need to invest in additional mechanical parts to help bind SortBot together. While this does increase the price of SortBot, hurting its economic feasibility, it is very much necessary to keep a robust safety level.

## 9.3 Public Welfare

With respect to public welfare, SortBot can contribute to leading workers of waste management facilities to more fulfilling roles. Workers that were once solely responsible for sorting trash can receive training in more technical or supervisory roles. However, what was once a team sorting

waste on a conveyor is now a single person watching over a monitor feed. To mitigate the displacement of workers, we added a feature to our web application that would allow multiple users to access the same video feed and change classifications labels. This way, it remains a team effort to properly sort trash.

# 10 RELATED WORK

## 10.1 AMP Delta

AMP's Delta robotic sorter has a very similar design and purpose. It's a robot made to sort recyclables from trash, which uses a suction-type end-effector to pick up objects and move them away from the conveyor belt. However, this robot uses the "Delta" gantry design, which uses three pairs of rods in a triangular shape that change in angle in order to move the end-effector where it needs to go.

## 10.2 Zen Robotics

Zen Robotics has made a waste sorting machine meant for lifting heavy-duty trash, such as wood and stone. They do use a linear gantry system, of using a suction-type end effector, they use a large, excavator-like claw to scoop and grab heavy items and throw them away from the main conveyor belt. Our design is similar, however it is intended for lightweight trash sorting.

# 11 SUMMARY

Our autonomous trash sorting robot is designed to decrease the overall amount of recyclables that end of in landfills. By having SortBot work on top of a conveyor belt we can simulate how it would work in waste management facilities. SortBot employs a deep learning model to localize the localize of recyclables, a 3-axis gantry to move to the location of a recyclable, and a barometer guage when we reach the recyclable. We were able to meet most of our design requirements. The biggest one we struggled on was achieving a pick-up and drop-off time of 8s. In the end we were only able to achieve an max of 18s. Given the limitations of the hardware we selected, it was difficult to reach the desired speed.

## 11.1 Lessons Learned

For student groups interested in pursuing a project that involves interacting with the physical world based on sensor reading or machine learning, we can not stress enough the importance of not underestimating the gap between theoretical assumptions and real-world implementation. What works in simulation or on paper often behaves very differently when exposed to noisy sensor data, hardware limitations, or unpredictable environments. The transition from concept to functioning prototype requires iterative testing, calibration, and a willingness to adapt both hardware and algorithms based on real-world feedback.

# Glossary of Acronyms

- COCO - Common Objects in COntext dataset
- CPU - Central Processing Unit
- DoF - Degrees of Freedom
- GFLOPS - Giga Floating Point OPerations Per Second
- GPU - Graphics Processing Unit
- I/O - Input/Output
- $mAP^{VAL}$ - mean Average Precision for VALidation dataset
- Mbps - Megabits per second
- MBps - MegaBytes per second
- mIoU - mean Intersection over Union
- OBB - Oriented Bounding Box
- ONNX - Open Neural Network eXchange
- RAM - Random Access Memory
- RGB - Red Green Blue
- SDK - Software Development Kit
- SSD - Single Shot multibox Detector
- TCP - Transmission Control Protocol
- TOPS - Time of Flight
- TOPS - Tera Operations Per Second
- UDP - User Datagram Protocol
- USB - Universal Serial Bus
- YOLO - You Only Look Once

# References

[1] Catalyst Planet. *US Lags Behind Other Nations in Recycling*. 2024. URL: `https://www.catalystplanet.com/travel-and-social-action-stories/us-lags-behind-other-nations-in-recycling` (visited on 05/02/2025).

[2] Cloudways. *Django vs Node.js: A Detailed Comparison*. Accessed: May 2, 2025. URL: `https://www.cloudways.com/blog/django-vs-nodejs/#performance-andscalability`.

[3] Roell Jr. *Python Flask vs Node.js Express*. `https://medium.com/@roelljr/python-flask-vs-node-js-express-4662b6f97b28`. Accessed: May 2, 2025. 2023.

[4] Tsung-Yi Lin et al. *Microsoft COCO: Common Objects in Context*. 2014. eprint: arXiv:1405.0312.

[5] Wei Liu et al. *SSD: Single Shot MultiBox Detector*. 2015. eprint: arXiv:1512.02325.

[6] Meuresiduo. *How Much Do We Waste? A Data-Driven Guide to Waste and Landfills*. 2022. URL: https://www.meuresiduo.com/blog-en/how-much-do-we-waste-a-data-driven-guide-to-waste-and-landfills/.

[7] Trash Dataset for Oriented Bounding Box. *Trash Detection Computer Vision Project*. 2023. URL: https://universe.roboflow.com/trash-dataset-for-oriented-bounded-box/trash-detection-1fjjc.

[8] Bhadresh Panchal. *Django vs NodeJS: Which is Better for Web Development?* 2024. URL: https://radixweb.com/blog/django-vs-nodejs.

[9] paulmelis. *opencv-camera-calibration*. URL: https://github.com/paulmelis/opencv-camera-calibration.

[10] Recycleye. *Retrofittable, robust and reliable 6-axis AI waste robots*. URL: https://recycleye.com/.

[11] Joseph Redmon et al. *You Only Look Once: Unified, Real-Time Object Detection*. 2016. eprint: arXiv:1506.02640.

[12] Mingxing Tan, Ruoming Pang, and Quoc V. Le. "EfficientDet: Scalable and Efficient Object Detection". In: (2019). eprint: arXiv:1911.09070.

[13] Matt Timpane. *In my opinion: MRF managers should focus on three areas*. Plastics Recycling Update. Sept. 2024. URL: https://resource-recycling.com/plastics/2024/09/05/in-my-opinion-mrf-managers-should-focus-on-three-areas/#:~:text=By%20accessing%20purity%20data%20in,40%20tons%20of%20aluminum%20cans (visited on 05/02/2025).

[14] Ultralytics. *Model Comparison: EfficientDet vs YOLOv8 for Object Detection*. URL: https://docs.ultralytics.com/compare/efficientdet-vs-yolov8/#performance-comparison-efficientdet-vs-yolov8.

[15] L. T. Worrell and J. A. Hensley. "A review of municipal solid waste environmental standards with a focus on incineration". In: *Journal of Environmental Health Science and Engineering* 16.2 (2018). Accessed via PubMed Central, pp. 407–416. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC6147112/ (visited on 05/02/2025).

[16] ZenRobotics. *ZenRobotics 4.0 - The Fourth Generation of our Waste Sorting Robots*. URL: https://www.terex.com/zenrobotics/4.0-efficiency-perfected.

[17] Zignuts. *Node.js vs Django: Which Backend Framework is Better?* https://www.zignuts.com/blog/nodejs-vs-django-backend-framework-comparison. Accessed: May 2, 2025. 2025.

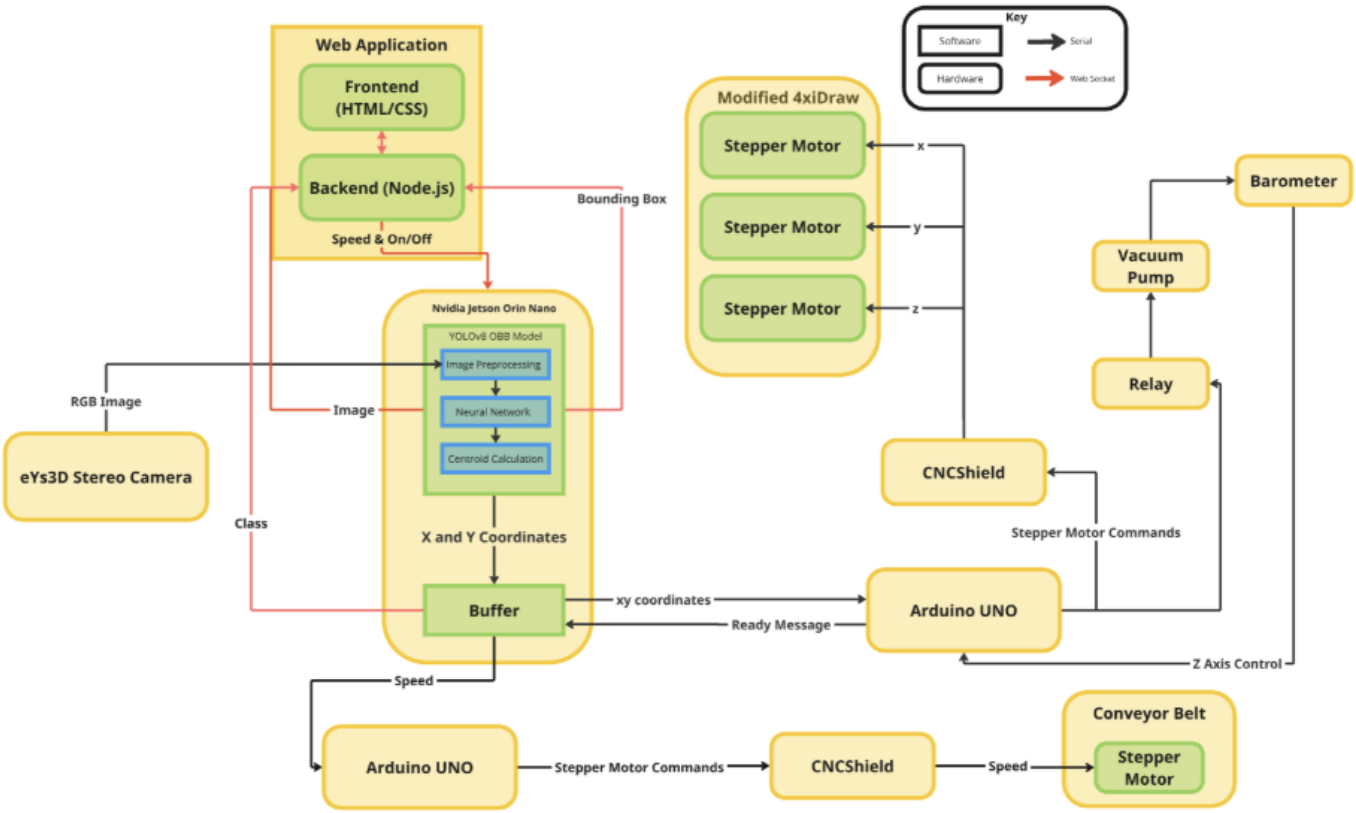


Figure 6: System Block Diagram

| Description | Manufacturer | Quantity in Package | Amount Purchased | Cost @ | Total |
|---|---|---|---|---|---|
| Nema 17 Steppers | STEPPERONLINE | 1 | 4 | $9.99 | $39.96 |
| 8 mm Smooth Rod | McMaster-Carr | 1 | 2 | $37.50 | $75.00 |
| LM8UU Bearings | MYCNCSHOP | 8 | 8 | $10.99 | $10.99 |
| 20-tooth GT2 Pulleys | WINSHINN | 2 | 1 | $6.99 | $6.99 |
| F623ZZ Bearings | uxcell | 10 | 10 | $8.99 | $8.99 |
| Arduino Mega 2560 Rev3 | Ardunio | 2 | 1 | $53.14 | $53.14 |
| CNC Shield and Two Pololu Stepsticks | ACERIRMC | 1 | 1 | $9.99 | $9.99 |
| GT2 Belt | SeekLiny | 1 | 1 | $9.99 | $9.99 |
| M10 Threaded Rods 1000 mm | McMaster-Carr | 2 | 4 | $14.86 | $29.76 |
| M10 Nuts | McMaster-Carr | 2 | 2 | $3.77 | $7.54 |
| M3 Screw Kit | Kadrick | 8 | 2 | $7.99 | $7.99 |
| 12V 2A Power Supply | NC | 1 | 1 | $7.99 | $15.98 |
| 1000mm 2020 aluminum extrusion | CtopoGo | 8 | 2 | 67.99 | 135.98 |
| Bearing Wheels | AFUNTA | 2 | 1 | $13.49 | $13.49 |
| Vacuum Cup | McMaster-Carr | 1 | 1 | $11.02 | $11.02 |
| Vacuum Pump DC 12V 12W | NC | 1 | 1 | $26.99 | $26.99 |
| Relays | AEDIKO | 4 | 1 | $6.99 | $6.99 |
| eYs3D Stereo Camera | eYs3D Microelectronics | 1 | 1 | $350.00 | $350.00 |
| Nvidia Jetson Orin Nano | Nvidia | 1 | 1 | $249.00 | $249.00 |
| Waste Bins | STERILITE | 4 | 1 | $24.99 | $24.99 |
| Silicone Tubing | DERPIPE | 1 | 1 | $10.99 | $10.99 |
| | | | | | $834.70 |

Table 4: SortBot's Bill of Materials

| TASK TITLE | START DATE | DUE DATE | % OF TASK COMPLETE |
|---|---|---|---|
| Deliverables | | | |
| Abstract | 1/15 | 1/22 | 100% |
| Website Setup | 1/25 | 2/1 | 100% |
| Project Proposal Slides | 1/26 | 2/2 | 100% |
| Project Proposal Presentation | 1/26 | 2/3 | 100% |
| Project Proposal Peer Review | 1/29 | 2/5 | 100% |
| Design Presentation Slides | 2/9 | 2/16 | 100% |
| Design Presentation | 2/10 | 2/17 | 100% |
| Design Peer Review | 2/12 | 2/19 | 100% |
| Design Document | 2/21 | 2/28 | 100% |
| Ethics Assignment | 3/5 | 3/12 | 100% |
| Interim Demo | 3/26 | 4/2 | 100% |
| Final Presentation Slides | 4/13 | 4/20 | 100% |
| Final Presentation | 4/16 | 4/23 | 100% |
| Final Presentation Peer Review | 4/16 | 4/23 | 100% |
| End of Semester Deliverables | 4/21 | 4/28 | 100% |
| Gantry XY Frame + Movement | | | |
| Purchase Main Components | 2/2 | 2/8 | 100% |
| Parts Assembly | 2/12 | 2/15 | 100% |
| Design changes + printing | 2/9 | 2/11 | 100% |
| Arduino Code XY Movement | 2/9 | 2/22 | 100% |
| Z/End Effector Movement | 2/23 | 3/1 | 100% |
| Jetson Nano I/O | 3/2 | 3/15 | 100% |
| Pick and Drop Sequence | 3/16 | 3/22 | 100% |
| Speed Optimizations | 3/22 | 3/29 | 100% |
| Machine Learning | | | |
| Research ML models and dataset | 2/3 | 2/8 | 100% |
| Dataset analysis | 2/5 | 2/8 | 100% |
| Setup video streaming pipeline | 2/10 | 2/16 | 100% |
| Write image processing pipeline | 2/16 | 2/24 | 100% |
| Create and train the ML model | 2/24 | 3/2 | 100% |
| Z-Axis Operator | | | |
| Purchase Electronic Components | 2/2 | 2/15 | 100% |
| Design Mechanics in CAD | 2/9 | 2/22 | 100% |
| 3D Print + Assembly | 2/16 | 2/22 | 100% |
| Mount to gantry + Arduino Code | 2/23 | 3/8 | 100% |
| I/O w/ Jetson Nano | 3/9 | 3/15 | 100% |
| Distance Sensor Code | 3/16 | 3/22 | 100% |
| Position Planning | 3/23 | 3/29 | 100% |
| Web App | | | |
| Frontend | 2/3 | 2/9 | 100% |
| Backend | 2/10 | 2/16 | 100% |
| Stream Video | 2/17 | 2/23 | 100% |
| Testing and Integration | | | |
| Integration | | | 100% |

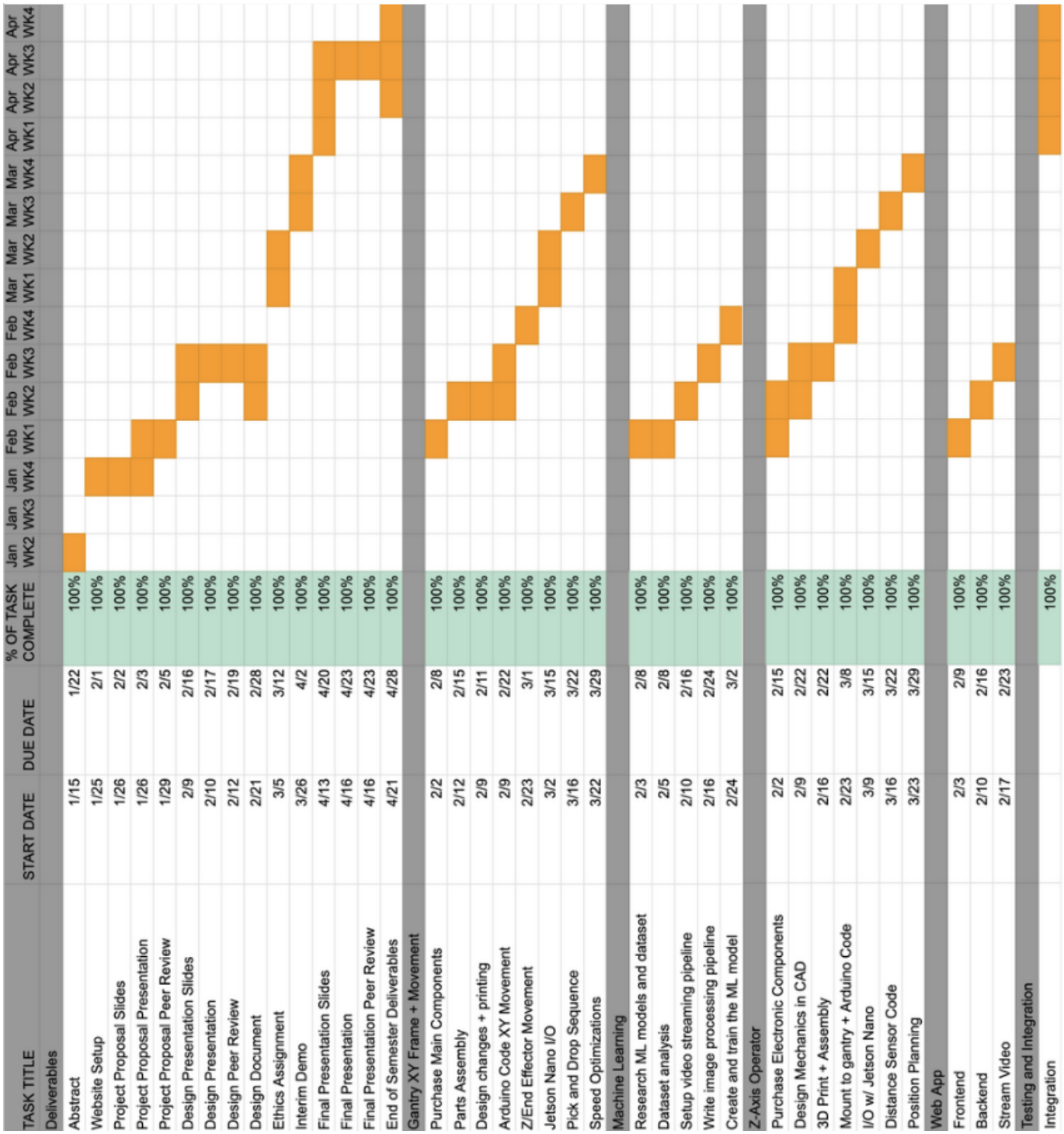


Figure 7: Gantt Chart